

Addestramento e Inferenza Distribuiti per Modelli Random Forest

Progetto Congiunto di Machine Learning e Sistemi Distribuiti e Cloud Computing

Gaia Meola
Matricola: 0369454

Università degli Studi di Roma "Tor
Vergata"
Roma, Italia
gaia.meola@alumni.uniroma2.eu

Sofia Tosti
Matricola: 0369460

Università degli Studi di Roma "Tor
Vergata"
Roma, Italia
sofia.tosti@alumni.uniroma2.eu

Emanuela Gallinelli
Matricola: 0386515

Università degli Studi di Roma "Tor
Vergata"
Roma, Italia
emanuela.gallinelli@alumni.uniroma2.eu

Sommario

Questo lavoro presenta un sistema distribuito Master-Worker per modelli Random Forest, in grado di operare sia in locale su container Docker sia in cloud su infrastruttura AWS. L'architettura supporta paradigmi di apprendimento sia centralizzati che federati e integra meccanismi avanzati di tolleranza ai guasti per assicurare la continuità operativa. La validazione condotta su benchmark reali e sintetici ne attesta l'elevata scalabilità computazionale e la piena neutralità algoritmica.

1 Introduzione

La scalabilità dei sistemi di Machine Learning rappresenta una delle sfide centrali dell'ingegneria del software moderna, dove la crescita costante dei volumi di dati si scontra con la capacità computazionale e di memoria dei singoli nodi: modelli fondamentali come le Random Forest [1] ne evidenziano severi colli di bottiglia se vincolati a un'esecuzione monolitica sequenziale. Le infrastrutture cloud-native forniscono le risorse computazionali necessarie, ma il reale fattore abilitante risiede in un'architettura distribuita capace di parallelizzare la costruzione degli alberi e la generazione delle predizioni su un cluster di nodi, accelerando i tempi di risposta senza alterare la qualità predittiva dell'ensemble. Su queste premesse, il presente lavoro illustra la progettazione, l'implementazione e la valutazione sperimentale di una piattaforma distribuita per modelli Random Forest, il cui codice sorgente è interamente disponibile nel repository pubblico dedicato https://github.com/tostfia/Distributed_RandomForest.git.

2 Struttura dell'Articolo

L'articolo è organizzato come segue: la **Sezione 3** descrive lo stack tecnologico, l'architettura modulare e i contesti di deployment locali e Cloud della piattaforma; la **Sezione 4** descrive l'architettura Master-Worker, con la separazione tra controllo e dati, e i paradigmi centralizzato e federato con le relative strategie di sharding. La pipeline di Machine Learning è dettagliata nella **Sezione 5**, sia per la classificazione su dataset reale sia per la regressione su dataset sintetico, mentre la **Sezione 6** illustra la progettazione infrastrutturale cloud-native su AWS, i relativi flussi di esecuzione e il ciclo di vita di un job, e la **Sezione 7** i meccanismi di resilienza basati su checkpointing incrementale e ripristino trasparente a fronte di crash dei nodi. Seguono la valutazione sperimentale rispetto alla baseline monolitica (**Sezione 8**) e la suite di testing e validazione funzionale (**Sezione 9**). Infine, la **Sezione 10** esamina le limitazioni dell'ambiente Cloud accademico e i possibili sviluppi futuri, prima delle considerazioni finali nella **Sezione 11**.

3 Piattaforma Software

L'intero sistema è stato sviluppato in **Python** tramite l'IDE Visual Studio Code, adottando un design architetturale modulare che disaccoppia la logica computazionale distribuita dai servizi infrastrutturali e dalle dipendenze software di terze parti.

3.1 Librerie di Terze Parti

L'infrastruttura software sfrutta i moduli nativi della libreria standard di Python ed un ecosistema selezionato di librerie specializzate. Comunicazione sincrona e orchestrazione remota tra Master e Worker sono affidate a **rpyc**, affiancato da **requests** per l'interazione via API HTTP con i servizi Cloud di stato e code. Il motore di Machine Learning per i modelli Random Forest si basa su **scikit-learn**, con **pandas** e **numpy** per caricamento e algebra matriciale, **scipy** per le funzioni statistiche

e il clustering gerarchico nella riduzione della multicollinearità, **pickle** per la persistenza dei modelli e **joblib** per la parallelizzazione della permutation importance in feature selection; a completare la pipeline concorrono **imbalanced-learn** per il bilanciamento del dataset e **optuna** per l'ottimizzazione degli iperparametri della baseline. L'interazione con l'ecosistema Cloud si appoggia agli SDK **boto3** e **botocore** per le primitive infrastrutturali e lo streaming dei dataset da Amazon S3, affiancati da **fsspec** per la scoperta dinamica dei file remoti e da **pyarrow** per il parsing accelerato dei CSV. Completano l'ecosistema **pydantic**, per la validazione formale delle strutture dati di scambio, **python-dotenv** per la gestione dinamica delle configurazioni via file d'ambiente, e la coppia **matplotlib/seaborn** per la generazione di report diagnostici e grafici prestazionali.

3.2 Ambienti di Esecuzione

Il sistema è progettato per garantire un'elevata portabilità e flessibilità di esecuzione attraverso differenti contesti operativi, configurabili tramite file d'ambiente (.env). Nello specifico, la **Baseline Monolitica** consiste in uno script sequenziale che esegue addestramento e inferenza del modello di Random Forest direttamente in locale sulla macchina host (WSL2 su Windows, con un processore Intel Core i7-13620H a 16 core e 15 GB di RAM allocati), fungendo da termine di paragone prestazionale assoluto. L'**Ambiente Locale Distribuito Nativo** esegue l'architettura avviando orchestratore e worker come processi concorrenti su terminali distinti mediante script dedicati, delegando a un supervisore la gestione della resilienza tramite logiche di *restart-on-failure* in assenza di containerizzazione. Parallelamente, l'**Ambiente Locale Containerizzato** sfrutta Docker Compose per isolare orchestratore e worker in container comunicanti via RPC su una rete bridge virtuale, con limiti espliciti di vCPU e RAM; in questo contesto i servizi di coordinamento sono emulati tramite mock su volume condiviso per riprodurre le API cloud senza dipendenze esterne, consentendo inoltre l'iniezione controllata di latenza e degrado di rete mediante tc-netem. Infine, l'**Ambiente Cloud AWS** rappresenta il contesto di produzione, il cui provisioning dichiarativo tramite Terraform traduce i blocchi logici in servizi gestiti nativi, descritti in dettaglio nella Sezione 6.

4 Architettura del Sistema

L'adozione di un'architettura **Master-Worker** garantisce il disaccoppiamento esplicito tra **piano di controllo** e **piano dati**, con ruoli e interazioni che restano invariati a dispetto dell'eterogeneità dei contesti operativi, differendo solo nell'implementazione dei moduli di supporto sottostanti. Nello specifico, il **Client** sottomette le richieste di addestramento e inferenza e ne monitora l'avanzamento senza intervenire nell'esecuzione; l'**Orchestratore (Master)** presidia il piano di controllo, ricevendo i job, scomponendoli in sotto-task e coordinandone l'assegnazione ai Worker, di cui traccia lo stato end-to-end; infine, i **Worker** costituiscono il piano dati, operando come nodi di calcolo dediti all'elaborazione intensiva per l'addestramento e l'inferenza. Il ciclo di vita di un job, con le relative transizioni di stato, è descritto in dettaglio nella Sezione 6 nel contesto della sua realizzazione su AWS, ma la medesima pipeline di coordinamento risulta identica anche in locale, dove gli equivalenti dei servizi gestiti replicano transizioni e responsabilità senza alterare la logica applicativa. Su questa base architetturale si innestano due regimi di esecuzione, selezionati staticamente all'avvio del sistema: nel paradigma **centralizzato**

i Worker sono nodi intercambiabili che operano su repliche dell'intero dataset o su partizioni assegnate dinamicamente dall'Orchestratore (Sezione 4.1), mentre nel paradigma **federato** ciascun nodo è vincolato al proprio dominio informativo locale [?], con l'Orchestratore limitato al coordinamento e all'assegnazione delle sotto-foreste da costruire (Sezione 4.2).

4.1 Modalità Operativa Centralizzata

All'interno della modalità centralizzata, la scelta di come distribuire il dataset ai Worker definisce due scenari contrapposti sul piano della scalabilità e della fedeltà teorica. La strategia **Shared** ricalca l'impostazione classica delle Random Forest distribuite [1, 2]: ogni Worker acquisisce l'intero dataset e ne estrae localmente i campioni per la propria quota di alberi, operando in modo analogo a un addestramento monolitico. Con dataset di grandi dimensioni, tuttavia, la replica integrale dei dati su ciascun nodo, combinata all'overhead di rete, può rendere la scalabilità distribuita persino meno performante della baseline monolitica (Sezione 8). Per superare questo collo di bottiglia è stata introdotta la strategia **Sharded**, basata sul partizionamento del dataset in **porzioni disgiunte**: pur introducendo un lieve scostamento teorico, dal momento che ogni albero viene addestrato su sottoinsiemi parziali anziché sull'intero dataset, essa garantisce un incremento prestazionale costante a fronte di una perdita minima sulle metriche di valutazione. La generazione degli *shard* varia in base alla natura del dataset: dinamica per quello sintetico, dove il numero di partizioni coincide con quello dei Worker attivi, e statica per quello reale, la cui numerosità campionaria finita renderebbe un partizionamento non controllato dannoso per la solidità statistica e le prestazioni di apprendimento dei singoli alberi. Va inoltre sottolineato che lo sharding centralizzato non va confuso con il paradigma federato: pur condividendo il principio del partizionamento, qui rappresenta una pura ottimizzazione infrastrutturale gestita a runtime dall'Orchestratore, che lascia i Worker anonimi e intercambiabili, capaci quindi di elaborare indifferentemente qualsiasi shard assegnato, a differenza del vincolo strutturale e localizzato tipico del Machine Learning federato. Il dataset di test resta, infine, unitario in entrambe le strategie: l'inferenza distribuita frammenta tra i nodi l'insieme degli alberi e non i dati, demandando a ciascun Worker la valutazione sull'intero dataset di test.

4.2 Modalità Operativa Federata

Nel paradigma federato, i dati nascono nativamente frammentati e dislocati sui singoli Worker: per il dataset reale, l'associazione tra Worker e shard viene stabilita in fase di *provisioning* offline e resta fissa per tutto l'esperimento, mentre per quello sintetico gli *shard* sono generati *in-memory* da ciascun Worker al primo task di addestramento, in base alla quota assegnata dall'Orchestratore. Per valutare l'impatto dell'eterogeneità dei dati, il partizionamento segue due topologie: la configurazione **IID**, ottenuta tramite *shuffling* uniforme dell'intero dataset [3], e la configurazione **Non-IID**, riservata al dataset reale, che sfrutta la struttura nativa del CSE-CIC-IDS2018 (Sezione 5.1) assegnando a ciascun Worker i CSV di interesse giornaliere di monitoraggio, preservando così lo sbilanciamento temporale e la concentrazione delle tipologie di attacco per emulare la naturale difformità del traffico tra domini di rete diversi. Su entrambe le distribuzioni si applicano due strategie di allocazione del budget di alberi: un'allocazione **uniforme**, con quote identiche per ogni nodo, e una **proporzionale** alla cardinalità dello shard locale, che traspone nei modelli ad albero il principio di FedAvg [4]. L'eterogeneità introdotta dallo sharding non-IID richiede strategie mirate: in aggregazione, per evitare che alberi addestrati su distribuzioni sbilanciate dominino l'ensemble, si adotta un *voto pesato per foglia*, basato sul numero di campioni confluiti in ciascuna foglia; analogamente, la soglia di decisione viene ricalibrata aggregando le quote di *validation set* locali, anziché ereditare la baseline centralizzata (usata solo in assenza di dati sufficienti). Anche l'inferenza segue una rigorosa segregazione: ciascun Worker valuta il modello globale solo sulla porzione di test del proprio shard locale, mai trasferita all'Orchestratore, che ne raccoglie unicamente le predizioni, da cui derivano due letture complementari delle metriche finali, una **pooled**,

sull'unione dei campioni e dunque dominata dagli shard più ampi, e una **macro-mediata**, che pesa ogni nodo allo stesso modo (Sezione 8.2).

5 Pipeline di Machine Learning

La valutazione sperimentale del sistema è strutturata su due scenari complementari, entrambi valutati sia in modalità centralizzata sia federata: il primo impiega il dataset reale **CSE-CIC-IDS2018** [5] per un task di **classificazione binaria** di *Network Intrusion Detection*, volto a misurare le prestazioni predittive del modello su dati complessi e sbilanciati; il secondo adotta un dataset sintetico generato tramite la funzione **make_friedman1** [6] di Scikit-Learn per un task di **regressione non lineare**, concepito come benchmark di scalabilità pura per isolare le prestazioni dell'infrastruttura dalla complessità del dato reale.

5.1 Dataset Reale: Campionamento

Il dataset di riferimento per la validazione su dati reali è **CSE-CIC-IDS2018** [5, 7], uno standard riconosciuto a livello internazionale per i sistemi di rilevamento delle intrusioni (*Intrusion Detection Systems*, IDS). Creato dal *Canadian Institute for Cybersecurity* (CIC) in collaborazione con il *Communications Security Establishment* (CSE), nasce dall'osservazione del traffico di una rete aziendale simulata ma altamente verosimile, distribuita su AWS per 10 giorni lavorativi, durante i quali sono state riprodotte sia le normali interazioni utente sia diversi vettori d'attacco (tra cui Brute Force, DoS/DDoS, Infiltration, Exploit Web). Il traffico grezzo, catturato in *pcap*, è stato elaborato per estrarre feature statistiche aggregate per sessione: ogni riga rappresenta un singolo flusso di comunicazione, caratterizzato da circa 80 parametri numerici e da un'etichetta che ne identifica la natura, benigna o malevola. Il dataset comprende 10 file CSV, uno per giornata di cattura, per un totale di circa 16 milioni di istanze, in cui il traffico benigno prevale nettamente (83%) su quello di attacco (17%). Nonostante l'ampia rappresentatività delle topologie di rete lo renda un solido benchmark, un'ispezione dei dati grezzi rivela tre criticità strutturali ampiamente documentate in letteratura [8–10], che ne impediscono l'uso immediato rischiando di indurre il modello ad apprendere scorciatoie mnemoniche e artefatti di rete anziché pattern d'attacco reali: un **troncamento dei dati**, con 7 file su 10 fermi esattamente a $2^{20} - 1 = 1.048.575$ righe (limite di esportazione tipico di Microsoft Excel); un marcato **sbilanciamento volumetrico tra le giornate**, con la sola giornata dedicata ai DDoS che assorbe quasi la metà del volume complessivo; e un profondo **sbilanciamento di classe interno**, con categorie come *Infiltration* ridotte a una minoranza quasi trascurabile. Lasciare inalterate queste distorsioni comprometterebbe sia la validità analitica dell'esperimento, generando un dataset sbilanciato e dominato dai giorni più voluminosi, sia la sua fattibilità operativa, data l'insostenibilità computazionale di addestrare l'intero volume su un unico nodo. Per superare questi vincoli si adotta un sotto-campionamento guidato, con una quota giornaliera di **200.000 istanze** e la semplificazione del task in una classificazione binaria tra traffico lecito e malevolo: una riduzione che costruisce un dataset bilanciato ed eterogeneo, rappresentativo delle diverse tipologie d'attacco, riconducendolo a una dimensione gestibile anche per la baseline monolitica in locale. Fa eccezione la gestione delle due giornate con attacchi *Infiltration* (Wednesday-28-02-2018 e Thursday-01-03-2018): data l'estrema rarità di questi campioni, che un campionamento casuale standard disperderebbe quasi del tutto, la pipeline preserva l'intera totalità del traffico non-Benign presente in questi due file (*Infiltration* ed eventuali altre categorie di attacco ivi presenti), applicando il sotto-campionamento al solo traffico lecito fino al raggiungimento della quota target.

5.2 Dataset Reale: Pre-Training

A seguito del campionamento guidato, il dataset consolidato di 2.000.025 istanze e 84 colonne viene sottoposto a una rigorosa pipeline di ingegneria delle feature e sanificazione, progettata per operare in isolamento e prevenire qualsiasi forma di *data leakage* [8]. In apertura si procede alla **binarizzazione del target**, mappando il traffico lecito nel valore 0 (Benign, 72,01% dei record) e le varie forme di attacco nel valore 1

(27,99%), quindi alla partizione in *train* e *test set* tramite `StratifiedShuffleSplit` di Scikit-Learn (`n_splits=1`) [3], che preserva le proporzioni originali tra le classi (1.600.020 istanze di addestramento, 400.005 di test). Esclusivamente a valle dello split, ciascuna partizione viene elaborata in isolamento attraverso quattro passaggi sequenziali: l'**isolamento topologico**, che rimuove sei metadati identificativi (`Dst Port`, `Timestamp`, `Flow ID`, `Src IP`, `Src Port`, `Dst IP`) vincolanti alla topologia di rete originaria; l'**ingegnerizzazione di tre metriche di dominio**, ovvero i coefficienti di variazione dei tempi di interarrivo (`Flow IAT CV` e `Fwd IAT CV`, rapporto σ/μ), che quantificano la *burstiness* del traffico indipendentemente dalla velocità scalare, e il rapporto `SYN_ACK_Ratio`, indicatore dell'asimmetria nel *three-way handshake* utile a individuare attacchi volumetrici o di scansione; e infine la **sanificazione numerica** dei valori non validi (`NaN/Inf`), posizionata deliberatamente in coda per intercettare anche i valori indefiniti generati dalle feature derivate (ad esempio i 465.751 `NaN` introdotti dalle due feature `CV` in presenza di denominatori nulli), al termine della quale il dataset di addestramento conta 1.142.662 istanze e 80 feature predittive, il *test set* 285.351 istanze. Prima del bilanciamento, un ulteriore 15% del dataset di addestramento (171.400 istanze) viene accantonato come *validation set* non contaminato per la calibrazione della soglia di decisione, riducendo il *train set* residuo a 971.262 istanze; su di esso si individua, a valle dell'addestramento, la soglia che massimizza l'*F1-Score* (0,7518), persistita nel manifesto del modello e riutilizzata sia dal percorso centralizzato sia da quello federato (Sezione 4), oltre a una soglia alternativa secondo il criterio di Neyman-Pearson, che massimizza la recall sotto il vincolo $FPR \leq 1\%$ (soglia = 0,6858). Soltanto al dataset di addestramento residuo si applica il trattamento dello sbilanciamento tramite **Random Under-Sampling** a rapporto 1 : 1 con le utilità di *imbalanced-learn*, coerentemente con le strategie di bilanciamento adottate in letteratura per il rilevamento delle intrusioni basato su Random Forest [11, 12], riducendo la classe maggioritaria (700.247 istanze Benign) fino a eguagliare la minoritaria (271.015 istanze Attacco) e portando il volume di addestramento effettivo a 542.030 istanze, mentre il dataset di test preserva integralmente la propria distribuzione originaria. La successiva **selezione delle feature**, condotta esclusivamente sul dataset di addestramento bilanciato e poi proiettata su test e validation set, riduce la dimensionalità in tre stadi a partire da 80 variabili: un primo filtro rimuove 8 variabili a varianza nulla (`Bwd PSH Flags`, `Bwd URG Flags`, `Fwd Byts/b Avg`, `Fwd Pkts/b Avg`, `Fwd Blk Rate Avg`, `Bwd Byts/b Avg`, `Bwd Pkts/b Avg`, `Bwd Blk Rate Avg`); sulle restanti 72 segue il calcolo della *OOB Permutation Importance* [1]¹ tramite una foresta preliminare (100 alberi, indipendente dalla configurazione finale tunata in Sezione 5.3), che non ne esclude alcuna (soglia = 0,0, tutte con aumento percentuale dell'errore OOB strettamente positivo); infine un *clustering gerarchico Ward linkage* sulla matrice di distanza di Spearman ($1 - |\rho_{\text{Spearman}}|$), secondo la metodologia illustrata nell'esempio ufficiale Scikit-Learn sulla gestione di feature multicollineari o correlate [13], tagliato alla soglia di 0,2 (Figura 1), individua 16 cluster con più di un membro e ne seleziona per ciascuno il solo rappresentante con importanza OOB più elevata, scartando 47 feature ridondanti. L'azione combinata dei tre filtri ($8 + 0 + 47 = 55$ feature eliminate) riduce lo spazio predittivo a sole **25 variabili** finali. Va precisato che questo risultato a 25 variabili è riferito al candidato `max_features='log2'`: come illustrato nella sezione seguente, la selezione viene infatti ricalcolata separatamente per ciascun valore di `max_features` esplorato nel tuning.

5.3 Dataset Reale: Tuning degli Iperparametri

Per la ricerca e l'ottimizzazione degli iperparametri della Random Forest è stata impiegata la stima **Out-Of-Bag (OOB)** in luogo della tradizionale *K-Fold Cross-Validation*, sulla base delle proprietà teoriche dimostrate da Breiman [1], secondo cui l'errore OOB fornisce una stima non distorta e accurata quanto quella su un *test set* indipendente, purché la foresta superi

¹Implementata secondo la formulazione originaria di Breiman (Sez. 10), tramite accesso diretto agli indici di campionamento bootstrap del modello, e non tramite l'utilità standard `sklearn.inspection.permutation_importance`. Quest'ultima è invece impiegata per la gestione della multicollinearità (metodologia dell'esempio ufficiale Scikit-Learn [13]).

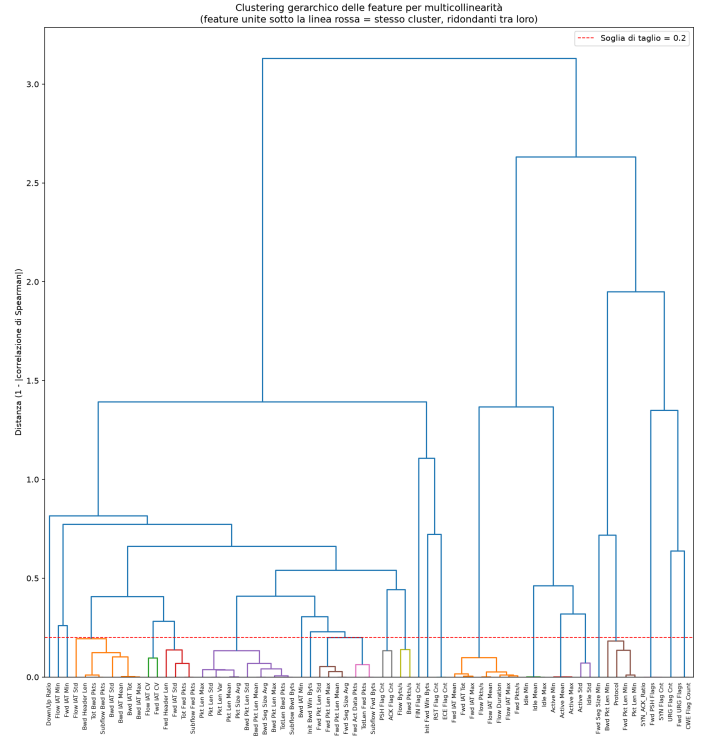


Figura 1: Dendrogramma del clustering gerarchico delle feature basato sulla distanza di Spearman per la riduzione della multicollinearità con soglia di taglio = 0.2.

la soglia di convergenza dell'errore, condizione verificata empiricamente.²³ A valle del campionamento e della sanificazione, la selezione delle feature descritta in Sezione 5.2 (16 cluster, 25 variabili finali) è riferita al candidato `max_features='log2'`, risultato vincente del tuning descritto di seguito; poiché `max_features` condiziona la foresta preliminare su cui è stimata l'importanza OOB, la stessa selezione è stata ricalcolata anche per `max_features='sqrt'`, producendo un secondo set di feature impiegato per i relativi trial. La ricerca è stata guidata dal framework **Optuna**, tramite il campionatore bayesiano **TPE (Tree-structured Parzen Estimator)**, con 60 *trial* sequenziali orientati a massimizzare l'*F1-Score* OOB sul dataset di addestramento di 542.030 istanze: rispetto a una Grid Search esaustiva sullo stesso spazio a quattro dimensioni effettive (`n_estimators`, `max_features`, `criterion`, `max_samples`; `max_depth` e `min_samples_split`, fissati a un solo valore, non contribuiscono allo spazio di ricerca), il campionamento Bayesiano-sequenziale raggiunge una stima affidabile dell'ottimo con un budget di trial molto più contenuto, orientando la ricerca verso le regioni più promettenti anziché valutare l'intero spazio in modo uniforme. Sono stati esclusi dallo spazio esplorato `max_depth` e `min_samples_split` (fissati rispettivamente a `None` e 2, combinazione raccomandata dalla guida ufficiale di Scikit-Learn [15] per foreste pienamente sviluppate) e `class_weight`, rimosso perché, sul train set già bilanciato 1:1 dall'undersampling a monte, `class_weight='balanced'` produce pesi indistinguibili da `None`, sottraendo così budget di trial a parametri realmente informativi. Un punto rilevante per la validità interna della ricerca è proprio questa integrazione della *feature selection* dentro la pipeline di tuning anziché a monte con una configurazione fissa: un

²Va segnalato che l'assunzione di non distorsione della stima OOB è stata in parte ridimensionata da studi successivi, che riportano una tendenza sistematica alla sovrastima dell'errore reale, particolarmente marcata in contesti di classi bilanciate [14], condizione che caratterizza il presente train set dopo l'undersampling 1:1 (Sezione 5.2). Gli stessi studi osservano tuttavia un impatto trascurabile di questa sovrastima sulla selezione degli iperparametri, che resta l'uso qui adottato; le metriche di generalizzazione riportate in Sezione 5.4 sono in ogni caso calcolate su un test set realmente indipendente, non sulla stima OOB.

³Si segnala inoltre un comportamento non documentato di Scikit-Learn 1.6.1 riscontrato empiricamente: le osservazioni prive di copertura OOB vengono riempite con probabilità [0,0] anziché con `NaN`, un bias che si propaga anche nell'attributo nativo `oob_score_`. La pipeline di stima adottata gestisce esplicitamente questo caso per evitare la contaminazione della metrica usata nel tuning.

set di feature calcolato una volta sola, prima del tuning, avrebbe favorito artificialmente uno dei due valori di `max_features` messi a confronto. La configurazione ottimale risultante è:

```
n_est = 200,      max_depth = None,   min_split = 2
max_feat = log2,   max_samp = 0,3,     criterion = entropy
```

con OOB F1-Score pari a 0,9109 e OOB Accuracy pari a 0,9142. Il valore di `n_estimators` è stato infine sottoposto a una verifica indipendente, volta a escludere che la scelta ricadesse in una regione ancora instabile della stima OOB (unica condizione, come detto in apertura, sotto cui la non distorsione è garantita): mantenendo fissi gli altri iperparametri, una singola foresta è stata fatta crescere incrementalmente tramite `warm_start=True` lungo una griglia da 5 a 200 alberi, misurando ad ogni checkpoint l'OOB error rate ($1 - \text{oob_score_}$), secondo la metodologia dell'esempio ufficiale Scikit-Learn "OOB Errors for Random Forests" [16]. Come mostrato in Figura 2, l'errore OOB decresce rapidamente fino a circa 100–110 alberi, oltre i quali la curva si stabilizza su un plateau senza ulteriori guadagni sistematici: il valore `n_estimators = 200` scelto dal tuning ricade in tale plateau, confermando a posteriori la non distorsione della stima OOB usata per selezionarlo.

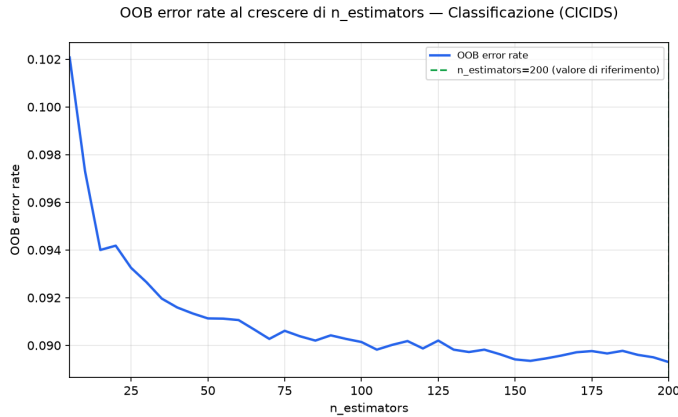


Figura 2: Andamento dell'OOB error rate ($1 - \text{oob_score_}$) al crescere del numero di alberi (`n_estimators`), ottenuto tramite approccio `warm_start` a parità degli altri iperparametri ottimali.

5.4 Dataset Reale: Risultati

La calibrazione descritta in Sezione 5.3 individua la soglia di decisione 0,7518, ottenuta massimizzando l'F1-Score sul validation set: un criterio alternativo di Neyman-Pearson (vincolo $\text{FPR} \leq 1\%$) produce sullo stesso validation set una soglia di 0,6858 con un F1-Score sul test pressoché equivalente ma un FPR quasi doppio (0,98% contro 0,49%) a fronte di un guadagno di recall marginale (circa un punto percentuale). Si adotta pertanto la soglia 0,7518 come riferimento per il resto del lavoro, coerentemente con la priorità di contenere i falsi allarmi tipica di un *Security Operations Center*; tale soglia viene persistita nel manifesto di configurazione e riutilizzata automaticamente dall'architettura distribuita, garantendo che il confronto tra paradigmi nelle sezioni successive avvenga a parità di *operating point*. Applicando la soglia 0,7518 al dataset di test (285.351 istanze), il modello raggiunge un'accuratezza del 94,59%, una precisione del 98,48% e una recall dell'81,85%, per un F1-Score complessivo dell'89,39%; il tasso di falsi positivi (FPR) si attesta allo 0,49%, mentre il ROC-AUC di 0,9778 conferma un'elevata capacità discriminativa indipendentemente dalla soglia adottata. La matrice di confusione in **Figura 3** evidenzia la distribuzione degli errori: sui 205.812 flussi benigni, appena 1.008 vengono classificati erroneamente come attacco (0,49%), mentre sui 79.539 flussi malevoli si registrano 14.439 mancati rilevamenti (falsi negativi). Tale asimmetria è la diretta conseguenza della soglia scelta, superiore al valore di default pari a 0,50: innalzare la soglia aumenta la selettività del modello, spingendo la precisione al 98,48% (a fronte

dell'87,35% ottenibile a soglia 0,50) a discapito di una flessione della recall di circa sei punti percentuali (81,85% contro 87,77%).

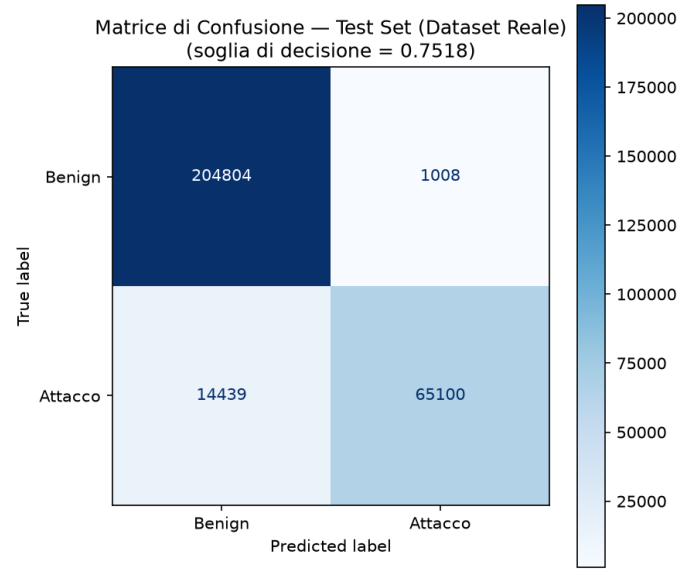


Figura 3: Matrice di confusione sul dataset di test del reale.

5.5 Dataset Sintetico: Generazione

Generato tramite la funzione `make_friedman1` [6] di Scikit-Learn, il dataset sintetico non lineare di riferimento per l'analisi della scalabilità è configurato con $N = 1.000.000$ istanze e $M = 100$ feature. Introdotto originariamente da Friedman (1991) e impiegato da Breiman (2001) [1] nei lavori fondativi sulle *Random Forest*, questo benchmark consente di isolare il calcolo puro e misurare la scalabilità in un regime in cui solo le prime cinque feature generano il target, mentre le restanti 95 agiscono da rumore controllato. In questa fase non è stata eseguita una ricerca sistematica degli iperparametri, preferendo una configurazione di baseline controllata caratterizzata da tre variazioni mirate rispetto ai default di `RandomForestRegressor` [17]: il parametro `max_features` è stato impostato a $M/3 \approx 33$, seguendo le indicazioni teoriche classiche per i problemi di regressione raccomandate da Hastie, Tibshirani e Friedman [18]; la dimensione dell'ensemble, inizialmente pari al valore standard di 100 alberi, è stata incrementata a `n_estimators = 400` a seguito di test empirici preliminari, poiché i tempi di esecuzione con il valore standard risultavano troppo ridotti per apprezzare appieno il guadagno prestazionale dell'architettura distribuita; infine, per incrementare la diversità tra i singoli stimatori, si è applicato un sotto-campionamento delle osservazioni pari a `max_samples = 0,3`. Per tutti gli altri parametri, gli alberi sono stati fatti crescere senza restrizioni di profondità (`max_depth = None`, `min_samples_split = 2`), adottando il criterio del squared error con campionamento bootstrap standard.

5.6 Dataset Sintetico: Risultati

La baseline locale è stata valutata sulla partizione di test indipendente, ottenuta tramite uno split del dataset sintetico descritto nella Sezione 5.5. Sul test set il modello ottiene un $R^2 = 0,768$, un errore quadratico medio $\text{MSE} = 6,9901$ (con $\text{RMSE} = 2,6439$) e un errore assoluto medio $\text{MAE} = 2,1093$. Tali valori confermano che il modello cattura efficacemente la relazione non lineare sottostante: l' R^2 indica che circa il 77% della varianza del target è spiegata dall'ensemble, mentre MSE e MAE risultano perfettamente coerenti con l'ordine di grandezza del target (media 14,41, deviazione standard 5,48). Come evidenziato dalla **Figura 4**, che pone a confronto i valori predetti e reali sul test set mediante un diagramma a densità *hexbin* per gestire la forte numerosità campionaria, la grande maggioranza delle predizioni si addensa regolarmente lungo la bisettrice

$y = x$, a conferma della solida capacità dell'ensemble di ricostruire la funzione target nella regione centrale della distribuzione. Si osserva tuttavia una moderata regressione verso la media: nella coda inferiore dei valori reali ($target < 0$) il modello tende alla sovrastima, mentre nella coda superiore ($target > 25$) si registra una sottostima, con scarti crescenti all'aumentare della distanza dal centro della distribuzione.

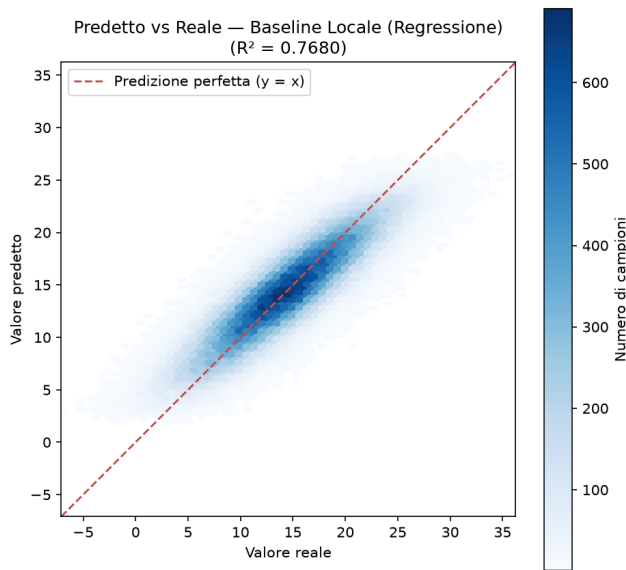


Figura 4: Diagramma *hexbin* dei valori predetti rispetto ai valori reali sul test set del dataset sintetico di regressione

6 Infrastruttura AWS

Riprendendo i principi dell'architettura di sistema delineata nelle sezioni precedenti, di seguito viene descritta la concreta declinazione su AWS delle relative componenti e dei flussi operativi associati. L'ambiente sperimentale adotta un'infrastruttura cloud codificata mediante **Terraform**, secondo il paradigma *Infrastructure as Code* [19]. I nodi Worker operano come **container serverless** tramite **Amazon ECS Fargate**, ciascuno configurato con 4 vCPU e 8 GB di RAM, mentre per il Nodo Orchestratore ci si affida a una coppia di istanze dedicate **Amazon EC2 r5.large**, con 2 vCPU e 16 GB di RAM, inserite in un **Auto Scaling Group** a capacità fissa. L'architettura disaccoppia il provisioning IaC dall'esecuzione: le risorse vengono predisposte a riposo e attivate dinamicamente a runtime tramite contatori indipendenti. Le principali interazioni tra i componenti sono schematizzate nella **Figura 5**, in cui la direzione delle frecce identifica l'iniziatore della comunicazione, ponendo così le basi per l'analisi dei flussi operativi approfondita nelle sezioni seguenti.

6.1 Ingresso del Job

La fase di sottomissione e presa in carico del job ha inizio quando il Client invia una richiesta **REST** verso **Amazon API Gateway** ①: l'API espone quattro rotte distinte (sottomissione di un job centralizzato, sottomissione di un job federato, interrogazione dello stato ed interrogazione dei dettagli), tutte instradate, tramite la stessa integrazione **AWS_PROXY** verso un'unica funzione **AWS Lambda**, ovvero **RFGatewayLambda** ②, che le distingue in base al metodo HTTP e al contenuto della chiamata. Le due rotte di interrogazione si differenziano per il livello di dettaglio restituito: quella di stato offre un riscontro minimale sufficiente a monitorarne l'avanzamento, mentre quella dei dettagli espone l'intera configurazione di sottomissione, pensata per consentire a un Client diverso da quello che ha avviato il training di recuperarla in un secondo momento (ad esempio per lanciare un'inferenza sullo stesso modello senza doverne già conoscere la configurazione). In fase di sottomissione, il Client specifica la **sorgente dati** (URL ad Amazon S3 o generazione di un dataset sintetico) e gli **iperparametri** del modello, configurabili ad hoc o ereditati dai default

derivati dalle fasi di tuning e calibrazione (vedi Sezioni 5.3 e 5.5). Ricevuta la richiesta, la Lambda registra il nuovo job nella tabella **ModelStatus** su **Amazon DynamoDB** ③ con stato **QUEUED**, quindi deposita il messaggio nella coda **Amazon SQS** di competenza (**centralized_queue**.fifo oppure **federated_queue**.fifo, con l'apposito **MessageGroupId**) e termina l'esecuzione ④, garantendo la persistenza delle richieste fino a 24 ore anche in assenza di orchestratori attivi. Il **job_id** assegnato in questa fase funge da identificativo del modello per eventuali richieste di inferenza successive, invocabili direttamente dal Client senza necessità di un identificativo separato.

6.2 Accodamento e Ripresa

L'Orchestratore Leader preleva i job dalla coda SQS via **long polling** ⑤, estendendo la visibilità del messaggio ogni 60 secondi durante l'elaborazione, per poi cancellarlo soltanto a compito ultimato. Affidare l'accodamento a SQS anziché a una distribuzione diretta dei job garantisce che ogni task rimanga tracciato e recuperabile anche in caso di crash del nodo ricevente, grazie al meccanismo nativo di **visibility timeout**: se il job non viene completato o rinnovato entro la finestra configurata, il messaggio torna automaticamente disponibile, pronto per essere reclamato da una nuova istanza o dal nodo subentrato in failover, indipendentemente dalla causa del fallimento precedente. Questo meccanismo non basta tuttavia a escludere l'elaborazione duplicata di uno stesso job: se la visibilità scadesse per un rallentamento anziché per un crash effettivo, un secondo Orchestratore potrebbe reclamare un messaggio il cui task è in realtà ancora in esecuzione. A prevenirlo interviene una **job lease applicativa** (tabella **JobLocks**), indipendente da SQS, approfondita nella Sezione 7.1. Preso in carico il job tramite la lease, l'Orchestratore ne porta lo stato a **PROCESSING** e avvia l'elaborazione, parallelizzando la costruzione della foresta in round assegnati ai Worker attivi. Se è invece il Leader stesso a interrompersi, il subentro non attende la scadenza del timeout SQS: l'elezione del nuovo coordinatore innesca un intervento immediato, che scansiona la tabella **ModelStatus** per intercettare e riprendere i task ancora in stato **PROCESSING**.

6.3 Dall'Orchestratore ai Worker

Preso in carico il job, l'Orchestratore ripartisce il carico tra i Worker disponibili e ne invia i sotto-task (i lotti di alberi da addestrare oppure i chunk di dati da valutare in fase di inferenza) via RPC su **porta TCP 18861** ⑥, all'interno di un traffico intra-cluster isolato e limitato ai soli membri del medesimo **Security Group**. Grazie alla rete **awsvpc**, che assegna a ogni task Fargate un IP privato distinto nella VPC, la coppia (IP, porta 18861) identifica univocamente ciascun Worker con un'unica porta fissa. Coordinazione e tolleranza ai guasti si appoggiano estesamente su **Amazon DynamoDB** ⑦, che ospita i lock di leadership e le lease sui job (**OrchestratorLocks**, **JobLocks**), lo stato delle sotto-task RPC (**WorkerTasks**), il progresso e i checkpoint dei job (**ModelStatus**), il contesto applicativo di ciascuno di essi (**JobMetadata**) e gli heartbeat di Orchestratori e Worker (**orchestrators_registry**, **workers_registry**), questi ultimi aggiornati ogni 10 secondi ⑧. Su questa base l'Orchestratore rileva ed esclude i nodi silenti da oltre due minuti, riassegnando o rinviando i relativi sotto-task secondo le politiche di tolleranza ai guasti (vedi Sezione 7). Esaurito il budget di alberi richiesto, l'Orchestratore ricompone infine la foresta, ne memorizza il manifesto e marca il job come **COMPLETED**, rimuovendolo dalla coda.

6.4 Piano Dati

La gestione del piano dati voluminoso fa capo ad **Amazon S3**, impiegato dall'Orchestratore per archiviare gli artefatti prodotti durante l'esecuzione del job ⑨. Nella modalità centralizzata, per evitare che più Worker scarichino ripetutamente lo stesso file, l'Orchestratore ne replica il dataset su un file system **Amazon EFS** condiviso nella VPC ⑨a, montato in sola lettura dai Worker Fargate e usato in via preferenziale, con ricaduta trasparente su S3 in caso di cache assente o mount fallito ⑨b. La modalità federata, priva di questa ridondanza da ammortizzare poiché ogni Worker legge un proprio shard esclusivo, non beneficia del meccanismo: qui S3 assolve una duplice funzione, fungendo sia da canale di scambio

dati per lo scaricamento degli shard e il deposito dei lotti completati sia da cache delle task già eseguite evitando il ricalcolo di elaborazioni precedenti ⁽¹⁰⁾. A seconda del regime operativo selezionato, in fase di inferenza l'Orchestratore distribuisce il test set o interroga gli *shard* locali, aggregando infine le predizioni e persistendo le metriche risultanti sullo storage condiviso, recuperabili dal Client con un'interrogazione successiva e indipendente. A job ultimato, il Client può inoltre scaricare il **modello addestrato** ⁽¹¹⁾: a differenza degli altri artefatti, il modello non esiste mai come file già assemblato su S3, bensì viene ricomposto on-demand direttamente dal Client, che accede allo storage dei checkpoint e recupera il manifesto leggero insieme alle singole parti degli alberi, per poi assemblarle localmente in un oggetto *scikit-learn* completo, serializzato in **formato pickle** e salvato su disco, direttamente utilizzabile con `model.predict()` senza dipendenze dal resto del sistema.

6.5 Componenti di Supporto

Infine, l'infrastruttura condivide i componenti di supporto per il ciclo di vita e l'osservabilità. Entrambi i ruoli prelevano la stessa **immagine Docker** da **Amazon ECR** ⁽¹²⁾, garantendo l'allineamento del codice pur specializzandosi dinamicamente all'avvio: i task Fargate dei Worker la scaricano tramite l'**agente ECS**, mentre le istanze EC2 dell'Orchestratore eseguono un `docker pull` nello script di boot. I log sono invece centralizzati su **Amazon CloudWatch** ⁽¹³⁾ tramite il driver *awslogs*, integrato nativamente via ECS per i Worker (`/ecs/lab-worker`) e a livello di demone Docker per l'Orchestratore (`/ec2/lab-orchestrator`).

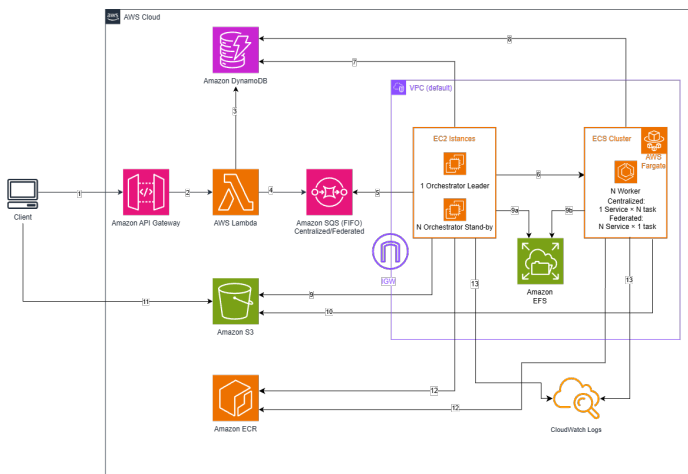


Figura 5: Componenti e flussi di comunicazione dell'architettura del sistema su AWS.

7 Tolleranza ai Guasti

L'addestramento di foreste di grandi dimensioni costituisce un'operazione computazionalmente onerosa e prolungata, in cui l'eventuale perdita dei progressi conseguiti, causata da un guasto hardware o da un'interruzione imprevista, si tradurrebbe in un dispendio inaccettabile di tempo e risorse. In un contesto distribuito, la tolleranza ai guasti non rappresenta pertanto una semplice ottimizzazione, bensì un requisito fondamentale per preservare la fattibilità stessa del processo di apprendimento. A tal fine, il sistema adotta un insieme di contromisure progettate per fare in modo che un guasto si traduca, nella peggiore delle ipotesi, in un mero rallentamento temporaneo e mai nella compromissione del lavoro svolto. Tali meccanismi operano trasversalmente su Worker e Orchestratore, beneficiando di un'implementazione coerente e simmetrica in entrambi gli ambienti di esecuzione. A fronte dell'interruzione o del crash di un nodo **Worker**, il pilastro architetturale della tolleranza ai guasti è rappresentato dal **checkpointing incrementale**: lo stato del modello non viene serializzato per intero a ogni passo, bensì per *delta*, registrando esclusivamente l'incremento computazionale prodotto da ciascun blocco di alberi completato su partizioni numerate. Un contatore monotono, infatti, sovrintende al corretto ordinamento temporale delle scritture,

scartando silenziosamente eventuali snapshot tardivi per prevenire la sovrascrittura di stati di avanzamento successivi. È questo meccanismo a consentire la ripresa del lavoro dal punto esatto dell'interruzione, evitando di ricominciare da capo in seguito alla sostituzione o al recupero del nodo compromesso. Su questa base, la capacità di **riassegnare il lavoro residuo** a un'unità alternativa varia tuttavia a seconda della modalità operativa: nel caso **centralizzato**, ogni Worker accede all'intero dataset o a shard intercambiabili, la riassegnazione è quindi sempre possibile. In addestramento avviene senza limiti di ritrasmissione, finché resta attivo almeno un Worker; in inferenza, superata una soglia configurabile di tentativi per sotto-task, l'Orchestratore esclude definitivamente il chunk corrispondente e prosegue con quelli già raccolti, restituendo un esito parziale basato su un ensemble ridotto ma statisticamente valido (un fallimento totale si verifica solo nel caso limite in cui nessun Worker abbia contribuito). Nel paradigma **federato**, invece, lo shard è l'unica copia dei dati presente su quel nodo: la riassegnazione diretta è quindi strutturalmente impraticabile. Il sistema gestisce questo scenario con una politica di attesa temporizzata, che esclude il carico pendente oltre un timeout configurabile, preservando la continuità dei nodi superstiti al prezzo della perdita di visibilità su quei dati. Se il guasto di un nodo **Worker** può essere assorbito grazie all'intercambiabilità dei task e alla ridondanza dei tentativi, lo stesso non vale per l'**Orchestratore**. Quest'ultimo non rappresenta un'unità di calcolo sacrificabile, bensì il fulcro della gestione dello stato del job, incaricato di coordinare l'assegnazione dei compiti e di sovrintendere alla persistenza dei checkpoint precedentemente descritti. Questa centralità funzionale lo eleva tuttavia a un evidente **Single Point of Failure** per l'intera infrastruttura. Per questo motivo il sistema adotta una **strategia di ridondanza** strutturata concretamente su **tre livelli** di raggio d'azione crescente. Il primo livello è quello di **processo**, governato da una politica di riavvio Docker finalizzata a ripristinare il servizio sulla medesima istanza a fronte di un crash applicativo. Tale configurazione differisce nei due ambienti per ragioni di natura operativa: in locale si adotta la modalità *on-failure*, che riavvia il container in automatico solo in caso di errore, evitando di riaccendere un'istanza arrestata deliberatamente in un contesto di test esplicito; in produzione su EC2 si impiega invece la direttiva *unless-stopped*, necessaria affinché il container sopravviva a un riavvio dell'intero host senza interventi manuali. Il secondo livello, cioè quello di **infrastruttura**, interviene laddove il riavvio del processo risulti insufficiente, avvalendosi di meccanismi esclusivi dell'ambiente cloud: da un lato, l'**EC2 Auto Recovery** gestisce i guasti a livello di hypervisor o hardware, ripristinando la macchina con identici identificativi e indirizzo IP; dall'altro, l'**Auto Scaling Group** provvede a sostituire le istanze terminate con un nuovo nodo, il quale ricava l'hostname dai metadati EC2 e si auto-registra privo di configurazioni statiche. È proprio quest'ultimo meccanismo a preservare la configurazione a doppia istanza come invariante strutturale e non come evento isolato: l'ASG garantisce infatti la presenza costante di due nodi attivi (Leader e Standby) anche a fronte di guasti successivi, scongiurando il rischio che una prima sostituzione lasci il sistema con un'unica istanza vulnerabile. Il livello **distribuito**, infine, presiede alla continuità dell'orchestrazione coordinando il subentro operativo: attraverso l'algoritmo di elezione del Leader e il ripristino dello stato condiviso (trattato in dettaglio nella Sezione 7.1), esso designa tempestivamente il nuovo coordinatore attivo tra le istanze rese disponibili, colmando l'assenza del nodo in autonomia, senza dover attendere il recupero dell'infrastruttura precedente.

7.1 Elezione del Leader

Il meccanismo di ridondanza descritto in precedenza garantisce che, alla perdita del Leader, esista sempre un'istanza Standby pronta a subentrare. Resta però da risolvere un problema distinto: **come** le due istanze si accordano su chi sia effettivamente il Leader attivo in un dato momento, evitando sia che restino entrambe inerti in attesa l'una dell'altra sia che entrambe si ritengano leader e operino in conflitto sullo stesso job. È questo il compito dell'**algoritmo di elezione** basato su *lease* [20] e implementato tramite **scritture condizionali atomiche su Amazon DynamoDB** (tabella `OrchestratorLocks`). Rispetto a un protocollo di

consenso più generale come Raft o Paxos, l'adozione di un meccanismo a lease costituisce una scelta deliberatamente più leggera, giustificata dalla scala ridotta del problema: con solo due istanze in gioco e un modello di accesso monoutente, non è necessario replicare uno stato condiviso tra più nodi con garanzie di quorum, essendo sufficiente una primitiva di mutua esclusione temporizzata (un token detenuto da un solo nodo alla volta, soggetto a scadenza automatica in assenza di rinnovo) già offerta nativamente dal key-value store che il sistema impiega comunque come registro di stato. Il meccanismo distingue due questioni concettualmente distinte, a ciascuna delle quali corrisponde un lock separato. La prima riguarda l'identità del Leader attivo in un dato istante ed è risolta tramite un unico record condiviso dotato di *Time-To-Live* (TTL) pari a 180 secondi, rinnovato ogni 10 secondi da un thread di *heartbeat* per l'intera durata di vita del Leader. Qualora il rinnovo si interrompa (in seguito a un crash o a un isolamento di rete) il record decade naturalmente allo scadere del TTL e la prima istanza Standby che ne rileva la scadenza ne acquisisce il possesso mediante una scrittura condizionata lato server (*ConditionExpression*), che rifiuta l'operazione qualora il lock risulti ancora valido, impedendo così che due istanze Standby possano prevalere simultaneamente. Nel caso di un arresto controllato (ad esempio in occasione di uno *scale-in* dell'Auto Scaling Group, segnalato tramite SIGTERM) il Leader rilascia, invece, il lock esplicitamente, sottraendo il subentro all'attesa della scadenza naturale del TTL. La seconda questione, non ridondante rispetto alla prima, riguarda l'effettivo stato di avanzamento del job dichiarato dal Leader: un processo può infatti permanere attivo, e continuare a rinnovare il proprio lock di leadership, pur avendo il thread di elaborazione bloccato, ad esempio a causa di un *deadlock* o di una chiamata RPC priva di risposta. A tale scopo, a ciascun job è associata una *job lease* indipendente dal lock di leadership, caratterizzata da un TTL più ampio (300 secondi, rinnovato ogni 60), dimensionato per coprire l'intera durata di un round di elaborazione anziché la sola verifica della vitalità del processo. La validità di tale lease è verificata dal thread a ogni round di scrittura, secondo le prassi consolidate in letteratura per questo scenario [21]; alla sua perdita, il processo si arresta indipendentemente dal fatto che continui a ritenersi Leader. Resta un caso limite che nessuno dei due lock, singolarmente, risolve in modo completo: un processo isolato da una partizione di rete (non crashato, ma semplicemente irraggiungibile) può continuare a ritenersi legittimo detentore della lease anche dopo che un altro nodo ne ha assunto il possesso. Si tratta del classico problema dello *zombie process*, per il quale la letteratura propone tipicamente l'adozione di una *fencing token*: un numero di sequenza che invalida, a livello di storage, le scritture provenienti da un detentore ormai superato. Il sistema non implementa esplicitamente tale pattern, ma ne ottiene comunque le garanzie sostanziali per due vie indipendenti. Da un lato, la piattaforma AWS sincronizza di default gli orologi delle istanze EC2 e Fargate tramite l'Amazon Time Sync Service [22], garanzia infrastrutturale su cui il sistema si appoggia senza necessità di configurazione esplicita dal momento che rende affidabile la validità temporale del TTL. Dall'altro, un contatore sequenziale sulla persistenza degli alberi impedisce che una scrittura tardiva, proveniente da un processo isolato, sovrascriva uno stato già avanzato oltre quello noto al mittente. Lo stesso scenario, con analoga mitigazione, si ripresenta nel failover basato su SQS: un messaggio la cui visibilità scade per un semplice rallentamento, anziché per un crash effettivo, può indurre un secondo Orchestratore a reclamare un job che il primo ritiene ancora di stare elaborando. Il subentro di un nuovo Leader richiede la ricostruzione del contesto del job senza ripartire da zero: ciò è reso possibile dall'isolamento del contesto applicativo di ciascun job in una tabella JobMetadata dedicata, dalla quale il nuovo Leader recupera rapidamente il payload originale. La ripresa dell'elaborazione è resa di fatto idempotente da un controllo articolato su due livelli: a livello di Orchestratore, prima di ridistribuire un sotto-task o rigenerare uno shard viene verificata l'eventuale preesistenza del relativo artifact sullo storage condiviso; a livello di Worker, un controllo analogo sul singolo task consente di recuperare un risultato già calcolato anziché rieseguirlo.

8 Valutazione Sperimentale e Discussione

La valutazione del sistema distribuito confronta metriche predittive, prestazionali e di resilienza dell'architettura con la baseline monolitica secondo un'analisi multidimensionale articolata nelle sottosezioni seguenti.

8.1 Neutralità Algoritmica

Questa dimensione verifica la qualità predittiva del modello distribuito rispetto alla baseline monolitica. Sul task di classificazione (dataset reale), i risultati in **Tabella 1** mostrano che l'architettura Centralizzata Shared replica pressoché esattamente le metriche del modello monolitico locale, confermando la neutralità algoritmica del sistema quando ogni albero viene addestrato sull'intero dataset. Il paradigma Centralizzato Sharded e l'Apprendimento Federato in regime IID mantengono entrambi uno scostamento contenuto su accuracy, F1 e AUC, sebbene con una differenza qualitativa: mentre il Federato IID conserva un profilo di precision e recall sostanzialmente in linea con la baseline, il Centralizzato Sharded mostra un trade-off più marcato tra le due metriche, verosimilmente legato a come la frammentazione del training set tra gli shard influenza la soglia decisionale del voto aggregato. Il partizionamento Non-IID per giorno di cattura (*by_day*) evidenzia, invece, la degradazione più marcata, in particolare nella variante Proporzionale, dove l'eterogeneità nella distribuzione delle classi tra i Worker compromette sensibilmente recall e AUC. La stessa dinamica è confermata anche sul task di regressione (dataset sintetico) in **Tabella 2**: il Centralizzato Shared replica la baseline con uno scarto marginale, coerentemente con il fatto che ogni albero vede l'intero dataset di training, mentre le strategie basate sul partizionamento mostrano uno scostamento più marcato e, soprattutto, di entità sostanzialmente equivalente tra loro; il Centralizzato Sharded e l'Apprendimento Federato degradano in misura pressoché identica rispetto alla baseline, con un errore quadratico medio che sale da 6.99 a circa 7.8 in entrambi i casi. Questa equivalenza rafforza l'interpretazione della causa sottostante: non è il paradigma di addestramento federato in sé a introdurre la degradazione, bensì la frammentazione del dataset tra i nodi, un effetto che si manifesta in modo comparabile ogniquale volta un singolo albero venga addestrato su una porzione soltanto dei dati anziché sull'intero training set, indipendentemente dall'architettura che governa la distribuzione.

8.2 Impatto dell'Eterogeneità dei Dati

Questa sezione misura l'impatto della distribuzione non-IID e delle strategie di allocazione del budget di alberi nel paradigma federato. Sotto il regime IID, le due strategie di allocazione producono risultati praticamente indistinguibili; il partizionamento **non-IID by day**, invece, introduce una criticità prestazionale marcata: la forte asimmetria nella distribuzione delle classi e nella concentrazione degli attacchi tra i nodi provoca un crollo di recall e F1-score, nonostante l'adozione del voto pesato per foglia in aggregazione. In questo scenario non-IID, la strategia di allocazione fa una differenza sostanziale: contrariamente all'intuizione derivata da FedAvg, l'allocazione proporzionale alla cardinalità dello shard locale peggiora le prestazioni rispetto a quella uniforme, anziché migliorarle ($F1 = 79,23\%$ contro $88,33\%$). Una spiegazione plausibile è che, dando più peso ai nodi con più dati, il modello finisce per adattarsi soprattutto alle giornate di traffico più numerose, a scapito dei pattern di attacco rari osservati solo dai nodi più piccoli. Il confronto tra lettura **pooled** e **macro-mediata** in **Tabella 3** permette di verificare questa spiegazione a livello di singolo nodo, anziché di sola media globale: sotto IID le due letture coincidono ($\Delta F1 < 0,01$), segno che tutti i nodi si comportano in modo simile; sotto Non-IID il divario cresce invece marcatamente, fino a $\Delta F1 = 15,5$ punti nella configurazione proporzionale. Poiché la lettura **pooled** è dominata dai worker con shard più ampio, questo scarto è coerente con l'ipotesi sopra: un sottoinsieme di nodi, quelli con meno dati e più pattern di attacco rari, ottiene prestazioni locali sostanzialmente peggiori di quanto la sola metrica **pooled** riportata in **Tabella 1** lasci intendere.

8.3 Scalabilità Distribuita

L'analisi valuta le prestazioni del sistema distribuito misurando lo *speedup* al variare del numero di nodi di calcolo impiegati, per caratterizzarne lo *strong scaling* sia in addestramento sia in inferenza (risultati sintetizzati nella **Tabella 4**). Prima di procedere al confronto vero e proprio, è opportuno isolare un caso particolare: alla configurazione $n_estimators = 400$, l'addestramento in ambiente WSL resta misurabile, ma il corrispondente tentativo di inferenza genera un errore di *out-of-memory*, che rende impossibile calcolare il relativo speedup rispetto alla baseline multicore. A differenza dei confronti che seguono, che quantificano quanto l'architettura distribuita sia più rapida, questo caso evidenzia una condizione in cui l'esecuzione su singola macchina non è affatto praticabile: il modello, ricostruito interamente in memoria per la fase di inferenza, eccede la capacità disponibile su tale macchina, laddove la distribuzione del carico tra i Worker consente invece di aggirare il vincolo. L'architettura distribuita offre quindi, in questo scenario, non un semplice guadagno di velocità ma una **capacità operativa** altrimenti irraggiungibile. Sebbene osservato qui in un unico scenario, l'argomento è generalizzabile: qualunque configurazione il cui fabbisogno di memoria ecceda quello disponibile su singola macchina (un modello più grande, un dataset più ampio, un ambiente locale più vincolato) ricade nella stessa categoria di problemi per cui la distribuzione del carico non è un'ottimizzazione, ma una vera e propria condizione abilitante. L'addestramento, al contrario, è stato eseguito con successo su singola macchina in ogni configurazione testata, rendendo possibile un confronto diretto con la baseline che rivela differenze marcate tra i tre paradigmi distribuiti (vedi **Figura 6**). Il paradigma **Centralizzato Shared** mostra un comportamento non banale: a 3 Worker il tempo di addestramento (1783,14s) risulta quasi doppio rispetto alla baseline monolitica (856,08s⁴, speedup 0,48×) e il pareggio si avvicina solo a 5 Worker (945,50s, ancora sotto la baseline) per essere raggiunto a 7 (718,07s, speedup 1,19×). Anche a 10 Worker lo speedup resta modesto (1,57×), con un'efficienza parallela stabilmente bassa sull'intero intervallo testato (tra 0,16 e 0,18) e senza un trend di miglioramento apprezzabile al crescere dei nodi. L'effetto è attribuibile all'overhead di comunicazione e replicazione dei dati intrinseco a questa strategia, in cui ogni Worker riceve l'intero dataset di addestramento: a basso numero di nodi tale costo non è ancora ammortizzato dal guadagno di parallelizzazione e ne domina il tempo totale di esecuzione. I paradigmi **Centralizzato Sharded** e **Federato** mostrano invece un comportamento radicalmente diverso: entrambi superano la baseline già a 3 Worker (608,98s e 468,05s rispettivamente, contro 856,08s), con uno speedup che raggiunge, a 10 Worker, 3,56× per lo Sharded e 11,50× per il Federato. Il confronto non è tuttavia a parità di risorse di calcolo: ciascun Worker Fargate dispone di 4 vCPU (Sezione 6), per cui a 10 Worker il cluster ha a disposizione 40 vCPU complessive contro le 16 della baseline. Normalizzando lo speedup per questo rapporto ($16/40 = 0,4$), il guadagno a parità di capacità di calcolo scende a 1,42× per lo Sharded e 4,60× per il Federato: quest'ultimo conserva quindi un vantaggio architetturale genuino anche dopo la normalizzazione, mentre per lo Sharded gran parte dello speedup grezzo è attribuibile alla maggiore quantità di risorse complessive, più che a un'effettiva efficienza di parallelizzazione. Un dato che rafforza questa lettura emerge già a 3 Worker, dove il cluster dispone di sole 12 vCPU, meno delle 16 della baseline: nonostante questo svantaggio, sia lo Sharded (1,87× normalizzato) sia il Federato (2,44× normalizzato) superano comunque la baseline monolitica, segno che il vantaggio osservato a bassa cardinalità di nodi non è un artefatto della maggiore potenza di calcolo complessiva. Il vantaggio sistematico del Federato è coerente con il fatto che ogni Worker addestra il proprio sotto-modello in completa autonomia, scambiando con il coordinatore solo i parametri aggregati e non i dati grezzi: l'assenza di sincronizzazione a grana fine durante il training elimina buona parte dell'overhead di comunicazione che penalizza invece lo Shared. Un ulteriore effetto composto, proprio della strategia di partizionamento, amplifica ulteriormente il divario: all'aumentare dei

⁴misura la sola chiamata `fit()`, esclusa la generazione del dataset sintetico. Il tempo riportato per l'architettura distribuita (`duration_seconds`) include invece l'intera pipeline, ETL compreso.

Worker si riduce non solo il numero di alberi assegnati a ciascun nodo, ma anche il volume di dati locali su cui ciascun albero viene addestrato; riduzione che, nel Federato, dove ogni Worker genera autonomamente la propria quota del dataset sintetico, si manifesta in modo più marcato che nello Sharded, spiegando perché il suo speedup superi persino quello lineare atteso rispetto al solo numero di nodi. Il divario tra le tre architetture si accentua ulteriormente in fase di inferenza (**Tabella 4**): a parità di 10 Worker, il throughput del Federato (4075,61 campioni/s) è quasi 8 volte quello del Centralizzato Shared (511,93 campioni/s) e più del doppio di quello Sharded (1771,54 campioni/s), confermando anche in fase di predizione il vantaggio strutturale del paradigma. Complessivamente, i risultati indicano che la strategia di partizionamento dei dati è determinante per ottenere uno scaling efficiente, sia in addestramento sia in inferenza, mentre la semplice replicazione dei dati su più nodi introduce un overhead di comunicazione che ne limita significativamente i benefici, specialmente a bassa cardinalità di Worker. Tale vantaggio in termini di efficienza non è tuttavia privo di contropartite: come discusso in Sezione 8.1, la stessa frammentazione del dataset che rende le architetture partizionate più scalabili è anche la causa principale del loro scostamento, seppur contenuto in regime IID, più marcato in presenza di eterogeneità tra i nodi (Sezione 8.2), dalla qualità predittiva della baseline monolitica. La scelta tra le architetture proposte richiede quindi di bilanciare, in funzione delle priorità applicative, il guadagno in termini di scalabilità con la sua ricaduta sulla neutralità algoritmica del sistema.

9 Validazione del Sistema

La correttezza dell'architettura è verificata tramite un modulo di test automatizzato, il `Test Engine`, eseguibile sia in locale sia su AWS a partire dalla stessa immagine Docker di Orchestratore e Worker. In locale gira come un ulteriore servizio containerizzato, avviato insieme ai container che va a esercitare. Su AWS, per non gravare sulle istanze dell'Orchestratore né introdurre un servizio permanente, viene eseguito su un'istanza EC2 `r5.large` temporanea: avviata su richiesta prima di una sessione di test, configurata per auto-terminarsi al termine dello scenario, con la stessa immagine condivisa via ECR e i log inoltrati a CloudWatch come nel resto dell'infrastruttura. La suite d'esame valida il sistema su cinque macro-aree. Sul fronte delle **prestazioni e scalabilità**, viene analizzata la scomposizione dei tempi e lo *strong scaling* al variare dei Worker, misurando lo *speedup* rispetto alla baseline monolitica in addestramento e inferenza. Segue la verifica della **resilienza di rete**, che valuta l'impatto di ritardi e perdite di pacchetti, tramite `tc-netem` in locale o osservando la latenza inter-nodo su VPC in AWS. Un terzo blocco di test riguarda la **tolleranza ai guasti applicativa**, e dimostra la capacità del sistema di recuperare i task senza perdere il progresso svolto. Il crash dei Worker (`ecs:StopTask` o `docker kill`) attiva la ridistribuzione del lavoro residuo. Il crash del Leader, forzato senza finestra di grazia via SSM Run Command con `docker kill`, attesta invece il subentro automatico di un nodo Standby e la ripresa dal checkpoint persistito, allo scadere del TTL delle lease. Infine, due proprietà architetturali avanzate completano il quadro di affidabilità. La **concorrenza e safety** viene saggiata facendo competere N nodi sincronizzati nella mutua esclusione della leadership, garantendo che ogni round esprima un singolo candidato vincente. La **resilienza infrastrutturale** (esclusiva per AWS) viene invece verificata abbattendo direttamente un'istanza EC2 (`ec2:TerminateInstances`), per controllare il ripristino della capacità computazionale da parte dell'Auto Scaling Group.

10 Limitazioni e Sviluppi Futuri

Realizzare concretamente il sistema e validarlo sperimentalmente ha fatto emergere specifici vincoli operativi e applicativi, la cui disamina definisce il perimetro entro cui inquadrare i risultati e traccia le direttrici per le evoluzioni future del sistema.

10.1 Limitazioni Ricontrate

Le limitazioni di maggior rilievo derivano principalmente dalle stringenti **Service Control Policy** imposte dall'ambiente **AWS Academy Learner Lab**, che hanno precluso un'automazione *Infrastructure as Code*

interamente nativa tramite Terraform e condizionato diverse scelte di deployment: il limite massimo di memoria per i task ECS (8192 MiB) ha impedito un'architettura interamente *serverless* su Fargate imponendo lo spostamento dell'Orchestratore su istanze EC2 dedicate, mentre l'assenza della *capability* Linux CAP_NET_ADMIN e, a sua volta, dei permessi per l'alternativa nativa AWS Fault Injection Simulator, ha ridotto lo scenario di simulazione di rete, sul ramo AWS, alla sola misura della latenza RPC osservata anziché alla sua alterazione controllata. Inoltre, l'inibizione di specifiche chiamate API ha richiesto la configurazione manuale preliminare di bucket S3 e gruppi di log CloudWatch, mentre l'impossibilità di definire ruoli IAM personalizzati ha costretto l'infrastruttura a un unico ruolo generico (LabRole). Nel coordinamento distribuito, l'assenza di un *fencing token* nativo a livello di storage ha richiesto di affidare la coerenza temporale della lease al protocollo NTP condiviso tra EC2 e Fargate, mitigando senza eliminare del tutto i rischi di sovrascrittura concorrente legati al TTL. Una limitazione di natura diversa riguarda, infine, l'API Gateway: essendo la gestione di utenti autorizzati e permessi fuori dal perimetro del lavoro, l'accesso all'API non prevede alcun meccanismo di autenticazione: una lacuna accettabile nel contesto sperimentale, ma da colmare qualora il sistema evolvesse verso un utilizzo reale.

10.2 Sviluppi Futuri

Sulla base dei limiti individuati, una prima evoluzione naturale del progetto è la migrazione verso un'infrastruttura Cloud standard che permetterebbe di ripristinare un'architettura completamente *serverless* e di introdurre un meccanismo di autenticazione esplicito tramite un servizio come Amazon Cognito. Un secondo sviluppo riguarda l'affiancamento, alla serializzazione nativa in *Pickle*, dell'esportazione in *PMML* (*Predictive Model Markup Language*), per guadagnare portabilità verso motori di inferenza non vincolati all'ecosistema Python. Infine, un terzo fronte, di natura più metodologica, riguarda l'impiego della stima *Out-Of-Bag* (OOB), oltre la fase di tuning, direttamente al posto di un dataset di test tradizionale per le predizioni finali.

11 Conclusioni

In conclusione, i risultati ottenuti confermano la validità dell'approccio proposto: l'adozione di architetture distribuite e l'impiego delle tecnologie cloud rappresentano un fattore determinante in molteplici contesti applicativi, tra cui il Machine Learning moderno, e la piattaforma sviluppata si propone come una soluzione flessibile, affidabile e scalabile per l'esecuzione distribuita di algoritmi complessi su cluster eterogenei trasparenti all'utente finale.

Riferimenti bibliografici

- [1] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [2] Erwan Scornet, Gérard Biau, and Jean-Philippe Vert. Consistency of random forests. *The Annals of Statistics*, 43(4):1716–1741, 2015.
- [3] Scikit-learn Developers. Scikit-learn StratifiedShuffleSplit Documentation. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedShuffleSplit.html.
- [4] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 1273–1282, 2017.
- [5] Canadian Institute for Cybersecurity. CSE-CIC-IDS2018 Dataset. <https://www.unb.ca/cic/datasets/ids-2018.html>, 2018.
- [6] Scikit-learn Developers. Scikit-learn make_friedman1 Documentation. https://scikit-learn.org/stable/modules/generated/sklearn.datasets.make_friedman1.html.
- [7] Joffrey L. Leevy and Taghi M. Khoshgoftaar. A survey and analysis of intrusion detection models based on CSE-CIC-IDS2018 big data. *Journal of Big Data*, 7(1):104, 2020.
- [8] Gints Engelen, Vera Rimmer, and Wouter Joosen. Troubleshooting an intrusion detection dataset: The CICIDS2017 case study. In *2021 IEEE Security and Privacy Workshops (SPW)*, pages 7–12. IEEE, 2021.
- [9] Arnaud Rosay, Florent Carlier, and Pascal Leroux. Network intrusion detection dataset analysis: CSE-CIC-IDS2018 issues and shortcomings. In *International Conference on Information Systems Security and Privacy (ICISSP)*, pages 401–408, 2021.
- [10] Daniel Arp, Erwin Quiring, Feargus Pendlebury, Alexander Warnecke, Fabio Pierazzi, Christian Wressneger, Lorenzo Cavallaro, and Konrad Rieck. Dos and don'ts of machine learning in computer security. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 3971–3988, 2022.
- [11] Richard Zuech, John Hancock, and Taghi M. Khoshgoftaar. Detecting web attacks using random undersampling and ensemble learners. *Journal of Big Data*, 8(1):75, 2021.

- [12] Zhewei Chen, Wenwen Yu, and Linyue Zhou. ADASYN-Random forest based intrusion detection model. In *2021 3rd Software Engineering and Machine Learning Conference*, pages 1–6, 2021.
- [13] Scikit-learn Developers. Permutation Importance with Multicollinear or Correlated Features. https://scikit-learn.org/stable/auto_examples/inspection/plot_permutation_importance_multicollinear.html.
- [14] Silke Janitz and Roman Hornung. On the overestimation of random forest's out-of-bag error. *PLOS ONE*, 13(8):e0201904, 2018.
- [15] Scikit-learn Developers. Scikit-learn User Guide. <https://scikit-learn.org/stable/>.
- [16] Scikit-learn Developers. OOB Errors for Random Forests. https://scikit-learn.org/stable/auto_examples/ensemble/plot_ensemble_oob.html.
- [17] Scikit-learn Developers. Scikit-learn RandomForestRegressor Documentation. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html>.
- [18] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Springer, 2009.
- [19] Amazon Web Services. Terraform on AWS. AWS Prescriptive Guidance, 2026. Ultimo accesso: Settembre 2026.
- [20] Cary G. Gray and David R. Cheriton. Leases: An efficient fault-tolerant mechanism for distributed file cache consistency. In *Proceedings of the 12th ACM Symposium on Operating Systems Principles (SOSP)*, pages 202–210, 1989.
- [21] Amazon Web Services. Leader election in distributed systems. <https://aws.amazon.com/it/builders-library/leader-election-in-distributed-systems/>.
- [22] Amazon Web Services. Set the time reference on your EC2 instance to use the local amazon time sync service. <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/configure-ec2-ntp.html>. Ultimo accesso: Settembre 2026.

A Appendice: Tabelle e Grafici di Dettaglio

Questa appendice raccoglie le tabelle numeriche di dettaglio e i grafici di scalabilità citati nel corpo dell’articolo.

Configurazione	ACC.	PRE.	REC.	F1	AUC
1. Baseline					
Monolitico Locale	94.59%	98.48%	81.85%	89.39%	97.78%
2. Centralizzato Distribuito					
Cluster Multi-Nodo <i>shared</i>	94.60%	98.47%	81.86%	89.40%	97.78%
Cluster Multi-Nodo <i>sharded</i>	94.45%	99.26%	80.71%	89.03%	97.62%
3. Apprendimento Federato (Partizionamento IID)					
IID + Proporzionale	94.27%	97.38%	81.66%	88.83%	97.38%
IID + Uniforme	94.27%	97.38%	81.66%	88.83%	97.38%
4. Apprendimento Federato (Partizionamento Non-IID)					
Non-IID (by_day) + Proporzionale	88.27%	78.30%	80.18%	79.23%	89.01%
Non-IID (by_day) + Uniforme	94.04%	97.26%	80.90%	88.33%	91.16%

Tabella 1: Confronto delle prestazioni di classificazione sul dataset CIC-IDS2018 per la verifica della neutralità algoritmica tra Baseline, Centralizzato e Federato.

Configurazione	MSE	RMSE	MAE	R ²
1. Baseline				
Monolitico Locale	6.9901	2.6439	2.1093	0.7680
2. Centralizzato Distribuito				
Cluster Multi-Nodo <i>shared</i>	7.0041	2.6465	2.1107	0.7675
Cluster Multi-Nodo <i>sharded</i>	7.8264	2.7975	2.2287	0.7402
3. Apprendimento Federato				
Federato	7.7643	2.7864	2.2196	0.7410

Tabella 2: Confronto delle prestazioni di regressione sul dataset sintetico per la verifica della neutralità algoritmica tra Baseline, Centralizzato e Federato.

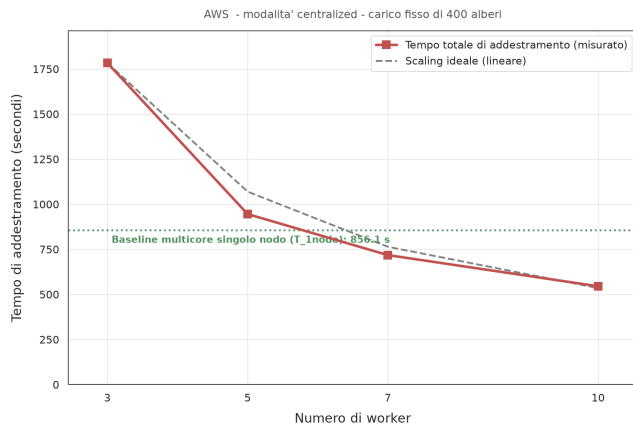
Configurazione	ACC.		PRE.		REC.		F1		AUC	
	P	M	P	M	P	M	P	M	P	M
IID + Proporzionale	94.27	94.27	97.38	97.39	81.66	81.66	88.83	88.83	97.38	97.38
IID + Uniforme	94.27	94.27	97.38	97.39	81.66	81.66	88.83	88.83	97.38	97.38
Non-IID + Proporzionale	88.27	85.36	78.30	68.18	80.18	66.53	79.23	63.72	89.01	79.99
Non-IID + Uniforme	94.04	90.55	97.26	89.06	80.90	75.68	88.33	75.84	91.16	89.08

Tabella 3: Confronto tra letture pooled (P) e macro-mediata (M) per le quattro configurazioni del modello federato.

Paradigma	Worker	Addestramento		Inferenza	
		Tempo (s)	Throughput (tree/s)	Tempo (s)	Throughput (samples/s)
Baseline (Multicore)	–	856.08	0.47	–	–
	3	1783.14	0.22	602.50	331.95
	5	945.50	0.42	441.22	453.29
	7	718.07	0.56	389.09	514.02
	10	545.03	0.73	390.68	511.93
AWS Centralizzato Shared	3	608.98	0.66	370.77	539.42
	5	365.49	1.09	177.67	1125.67
	7	273.27	1.46	134.32	1488.94
	10	240.74	1.66	112.90	1771.54
AWS Centralizzato Sharded	3	608.98	0.66	370.77	539.42
	5	365.49	1.09	177.67	1125.67
	7	273.27	1.46	134.32	1488.94
	10	240.74	1.66	112.90	1771.54
AWS Federato	3	468.05	0.85	86.81	2303.81
	5	187.95	2.13	63.92	3128.85
	7	108.93	3.67	54.06	3699.44
	10	74.42	5.37	49.07	4075.61

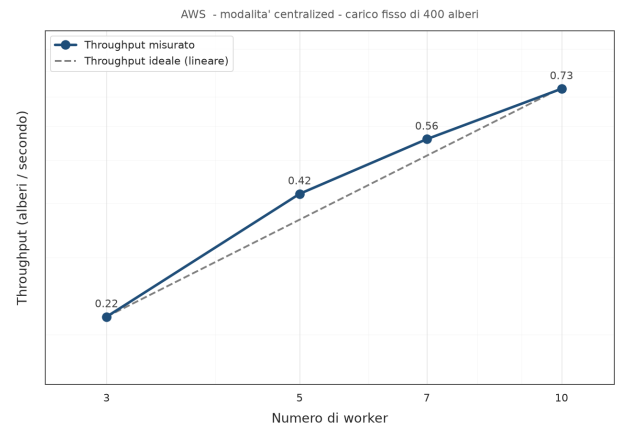
Tabella 4: Confronto delle prestazioni di addestramento e inferenza sul dataset sintetico al variare del numero di worker.

Strong scaling: tempo di addestramento a carico costante — Shared



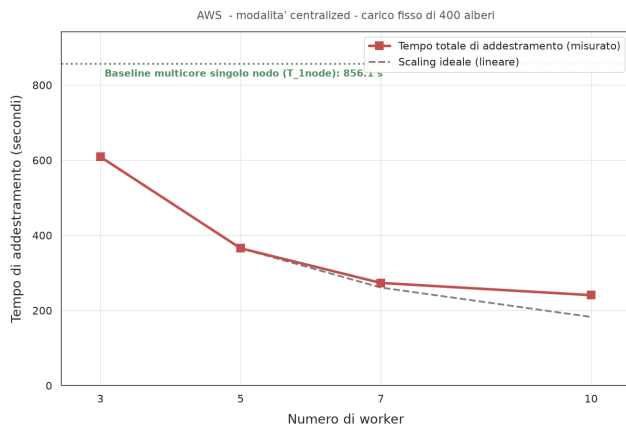
(a) Strong scaling - Centralizzata shared

Throughput addestramento del sistema distribuito — Shared



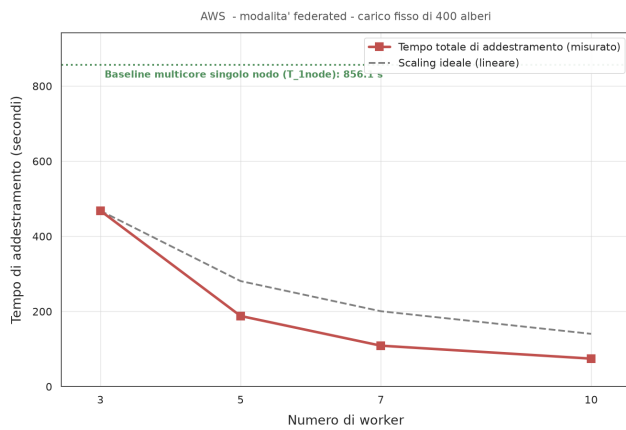
(b) Throughput - Centralizzata shared

Strong scaling: tempo di addestramento a carico costante — Sharded



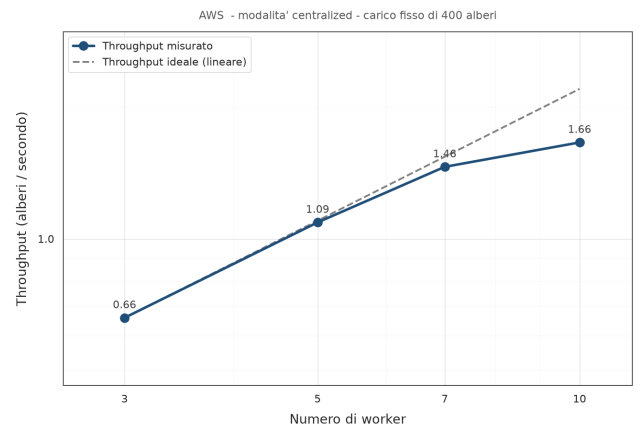
(c) Strong scaling - Centralizzata sharded

Strong scaling: tempo di addestramento a carico costante



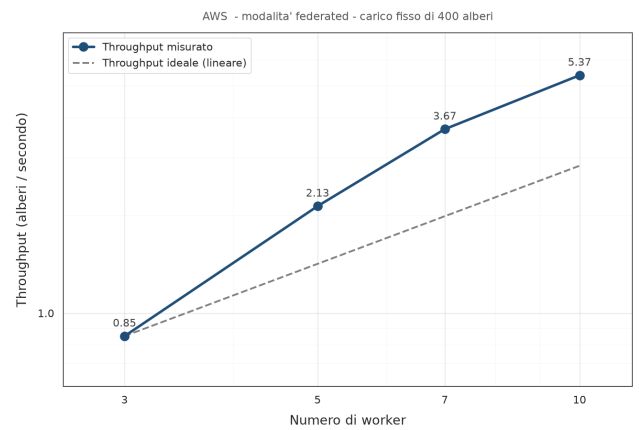
(e) Strong scaling - Federated

Throughput addestramento del sistema distribuito — Sharded



(d) Throughput - Centralizzata sharded

Throughput addestramento del sistema distribuito



(f) Throughput - Federated

Figura 6: Confronto di Strong Scaling e Throughput per le modalità distribuite su AWS.